

QUANTSHIELD

DevOps Continuous Integration, Delivery & Observability Report

PROJECT FOCUS

A practical DevOps layer around the existing QuantShield FastAPI backend, covering CI/CD, configuration management, containerization, Kubernetes orchestration, monitoring, and operational rollback.

Item	Details
Project	QuantShield — AI-powered quantitative portfolio risk monitoring
Primary Service	FastAPI backend served with Uvicorn
CI/CD	GitHub Actions
Configuration Management	Ansible
Containerization	Docker
Orchestration	Kubernetes (KinD validation cluster)
Monitoring	Prometheus + Grafana
Academic Context	BTech / PBL DevOps CA2

Executive Summary

QuantShield is an AI-powered financial risk monitoring application whose core service is implemented as a FastAPI microservice. This report documents the DevOps layer built around that existing service to establish a repeatable software delivery and operations workflow. The implementation combines GitHub Actions for continuous integration and delivery, Ansible for configuration management, Docker for immutable packaging, Kubernetes for orchestration and controlled rollouts, and Prometheus with Grafana for application observability.

The central design principle is to automate validation as early as possible while preserving the underlying quantitative and machine-learning logic. The pipeline verifies the API, validates infrastructure automation, builds the application image, and uses an ephemeral Kubernetes environment to validate deployment behavior. Operational readiness is reinforced with Kubernetes health probes and application metrics, while rolling-update and rollback demonstrations provide evidence of controlled change management.

CORE OUTCOME

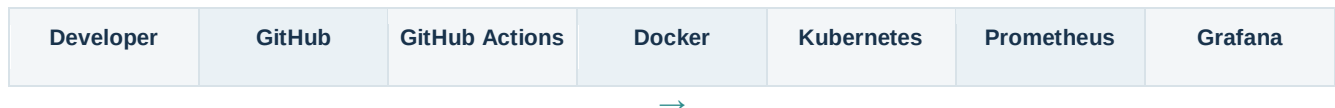
The result is a student-readable but professionally structured DevOps lifecycle that turns an existing ML application into a reproducible, testable, observable deployment artifact.

Report Structure

Section	Focus
1. System Architecture Overview	Target architecture and role of each DevOps component
2. CI/CD Pipeline Flow	GitHub Actions stages and validation sequence
3. Configuration Management & IaC	Ansible provisioning and idempotent runtime setup
4. Containerization & Orchestration	Docker image, Kubernetes deployment, rolling updates and rollback
5. Monitoring & Logging	Prometheus scraping, application metrics and Grafana dashboard
6. Engineering Challenges	Key deployment constraints and solutions
7. Lessons Learned	Operational and engineering takeaways
8. Validation & Evidence	Proof required for each assignment step
9. Artifact Index	Repository structure and submission references

1. System Architecture Overview

The QuantShield DevOps implementation establishes a continuous delivery and operations workflow around the existing FastAPI backend. The application ingests portfolio weights, retrieves historical market data, derives quantitative risk features including volatility, skewness, kurtosis, diversification ratio, and EWMA-based indicators, and produces continuous risk forecasts and calibrated tail probabilities. The DevOps design intentionally leaves the underlying risk and machine-learning logic unchanged while adding a controlled path from source code to a monitored Kubernetes workload.



The target architecture is composed of five functional layers: source control, automated validation, immutable packaging, orchestration, and observability. GitHub acts as the system of record. GitHub Actions executes the validation workflow on an ephemeral Ubuntu runner. Docker packages the service as an immutable image. Kubernetes runs two replicas and governs availability during changes. Prometheus and Grafana provide the operational feedback loop.

Layer	Technology	Purpose
Source control	GitHub	Stores application and DevOps artifacts and triggers automation on code changes.
CI/CD	GitHub Actions	Runs automated tests, infrastructure validation, image builds, and Kubernetes checks.
Configuration	Ansible	Creates the expected runtime configuration in an idempotent, declarative form.
Containerization	Docker	Packages the FastAPI service and its Python dependencies into a reproducible runtime image.
Orchestration	Kubernetes / KinD	Runs multiple replicas and validates deployment, rolling update, and rollback behavior.
Observability	Prometheus + Grafana	Collects and visualizes application availability, latency, traffic, and errors.

DESIGN PRINCIPLE

Shift-left validation: defects in application behavior, infrastructure automation, packaging, and rollout logic are detected before the deployment is considered successful.

2. CI/CD Pipeline Flow

GitHub Actions is used as the deployment automation tool required by the assignment. The workflow is event-driven and is designed to validate the application and its deployment artifacts before they are treated as releasable. Commits to the configured work/main branches trigger the workflow, while pull-request events provide an additional validation path for changes.

1. **Source control trigger.** A push or pull request starts the GitHub Actions workflow on the configured branch.
2. **Automated testing.** A clean Python 3.11 runner installs project dependencies and executes the API test suite. The checks cover the root endpoint, the risk service health endpoint, input validation for portfolio weights, and the instrumented /metrics endpoint.
3. **Ansible validation and provisioning.** The workflow installs Ansible Core, performs syntax/lint checks as configured, and runs the playbook against the provided inventory. The verified run produces the expected runtime structure, user/group configuration, environment file, and service definition.
4. **Container image build.** Docker Buildx packages the application using the repository Dockerfile. Dependency layers are separated from source-code layers so that repeat builds can reuse cached layers where possible.
5. **Kubernetes validation.** A lightweight KinD cluster is created on the runner. The built image is loaded, manifests are applied, and the deployment is checked for successful rollout and rollback behavior.

Push / PR	Tests	Ansible	Docker Build	KinD Deploy	Rollout / Rollback
-----------	-------	---------	--------------	-------------	--------------------



A notable strength of the pipeline is that deployment behavior is tested rather than only syntax-checked. A green workflow therefore represents more than a successful source-code build: it indicates that the service can be packaged and exercised in a Kubernetes environment with the required health and rollout controls.

3. Configuration Management & Infrastructure as Code

Ansible is used to represent the runtime environment as declarative configuration. The playbook targets the configured host inventory and creates a consistent application baseline without requiring individual manual setup steps. The implementation provisions a dedicated QuantShield system group and service user, creates the application directory structure, writes the runtime environment configuration, and registers the service definition used by the host environment.

Configuration Area	Implemented Action	Operational Benefit
Packages	Ensures required runtime and container tooling is installed.	Reduces dependency on manual host preparation.
Identity	Creates a dedicated QuantShield group and unprivileged service user.	Separates application execution from administrative identity.
Filesystem	Creates /opt/quantshield/{app,config,logs,models}.	Provides a predictable runtime layout.
Permissions	Applies controlled directory permissions.	Makes access behavior explicit and repeatable.
Runtime configuration	Writes the required environment configuration.	Centralizes deployment-time settings without embedding secrets in source code.

Service management	Generates the persistent systemd service definition.	Provides a standardized host-level service lifecycle.
--------------------	--	---

VERIFICATION

The recorded Ansible execution completed with the expected state and no failures (ok=9, changed=5, failed=0). Re-running the playbook is intended to converge on the same configuration rather than produce uncontrolled changes.

4. Containerization & Orchestration

Docker packages the QuantShield FastAPI backend with a Python 3.11-slim runtime. The image is structured so that dependency installation can be cached independently from application source changes. This reduces unnecessary rebuild work while keeping the runtime image focused on the components required to execute the service.

The Kubernetes deployment runs two QuantShield replicas behind a NodePort service. The deployment uses readiness and liveness probes against the application health endpoint, allowing Kubernetes to distinguish between a starting model process and a genuinely unhealthy runtime. This is particularly important for an ML-backed API because model deserialization can take longer than a conventional web process startup.

Control	Configuration	Purpose
Replicas	2 QuantShield pods	Provides basic service redundancy during normal operation and updates.
Service	NodePort, application port 8000 mapped through the configured node port	Provides stable access to the backend workload in the local Kubernetes environment.
Readiness probe	/api/v1/risk/health	Prevents traffic from reaching a pod until the service is ready.
Liveness probe	/api/v1/risk/health	Allows Kubernetes to restart an unhealthy container.
Update strategy	RollingUpdate	Replaces pods progressively instead of stopping the entire deployment.
Rollback	kubectl rollout undo	Restores the previous deployment revision when a release must be reverted.

Rolling Update and Rollback

The deployment lifecycle was demonstrated using two application image revisions. The update path changes the running deployment from v1.0.0 to v2.0.0 using Kubernetes rolling-update behavior, after which the rollout is verified. A rollback is then triggered with kubectl rollout undo and the restored revision is validated. This demonstrates that release changes are reversible and that the cluster retains deployment history rather than relying on manual recovery.

v1.0.0	Rolling update	v2.0.0	Rollback	v1.0.0 restored
--------	----------------	--------	----------	-----------------

**OPERATIONAL EFFECT**

With maxUnavailable=0 and the readiness gate in place, a healthy existing replica is retained until the

replacement becomes ready. This is the core mechanism used to demonstrate controlled, low-disruption application updates.

5. Monitoring & Logging

QuantShield exposes application metrics through the Prometheus instrumentation integrated into the FastAPI service. The `/metrics` endpoint publishes request-oriented telemetry in a standard OpenMetrics-compatible format. Prometheus then scrapes the service at a fixed interval, while Grafana queries Prometheus to present an operational dashboard.

<code>/metrics</code>	Prometheus	PromQL	Grafana dashboard
-----------------------	------------	--------	-------------------



Signal	Source	What it shows
Availability / uptime	Prometheus scrape health / up signal	Whether the monitored service is currently reachable by the monitoring system.
Latency	FastAPI request-duration metrics	How long API requests take and how response time changes over time.
Traffic	HTTP request counters	Request volume and throughput, including status-code breakdown where available.
Errors	HTTP status-code metrics	The frequency and proportion of failed requests such as 4xx and 5xx responses.

The dashboard is organized around the operational signals most useful for a small service: availability, latency, throughput, and errors. Synthetic test traffic may be generated during demonstrations to produce visible time-series data; the values are still collected from the running service rather than fabricated in the dashboard itself.

WHY OBSERVABILITY MATTERS

A deployment is not complete merely because the process is running. Metrics make the runtime state visible and provide evidence that the service remains healthy under normal requests and controlled failure cases.

6. Engineering Challenges and Solutions

1. Heavy scientific dependencies in Docker

Packages such as CatBoost, scikit-learn, pandas, and NumPy increase build size and build time. The implementation uses `python:3.11-slim`, removes unnecessary package-manager caches, keeps build-only tooling out of runtime layers where possible, and copies requirements before application source so dependency layers can be reused.

2. Slow model initialization and false pod restarts

ML models require a short cold-start period to load weights into memory. Aggressive health checks can therefore create crash-restart loops. The implementation separates readiness from liveness and adds startup delay so that traffic is held until the service is ready while avoiding premature restarts.

3. Live observability without code bloat

Manual instrumentation across every API route would add repetitive code and make the application harder to maintain. The `prometheus-fastapi-instrumentator` package provides automatic request counters, latency histograms, and related process/request metrics through a very small configuration change.

4. Reproducible deployment behavior

An ML service includes more than source code: dependencies, runtime configuration, model files, health behavior, and orchestration rules all influence the result. Treating environment setup, container packaging, and Kubernetes configuration as version-controlled artifacts makes the deployment process repeatable and inspectable.

7. Lessons Learned

- **Immutable infrastructure.** Containerizing the service establishes a consistent runtime boundary and reduces differences between development and deployment environments.
- **Idempotency is essential.** Ansible automation should converge safely when executed repeatedly. This turns configuration management into a controlled state-management process rather than a sequence of one-off shell commands.
- **Rollbacks are part of deployment design.** A release process is more reliable when reverting to the previous known-good version is a built-in operation rather than an emergency reconstruction.
- **Observability is an architectural concern.** Metrics must be considered as part of the service contract. Prometheus and Grafana convert runtime behavior into information that can be inspected, compared, and acted upon.
- **ML deployment needs different operational thinking.** Compared with a simple web API, an ML service has model-loading time, heavier dependencies, and a larger runtime footprint. These characteristics directly affect image construction and Kubernetes probe design.

8. Validation & Evidence Strategy

The assignment is evaluated not only on the existence of configuration files but on demonstrated execution. For that reason, the submission should retain both repository artifacts and runtime evidence. Each screenshot should prove one clear claim and should be named consistently with the corresponding assignment step.

Assignment Step	Implementation Artifact	Runtime Proof
Step 1 — CI/CD (1.0)	<code>/.github/workflows/ci-cd.yml</code>	Successful GitHub Actions workflow with visible green stages.
Step 2 — Ansible (2.0)	<code>/ansible/playbook.yml</code> and <code>/ansible/inventory</code>	Successful ansible-playbook execution and verified configured state.
Step 3 — Docker + Kubernetes (1.5)	<code>/Dockerfile</code> , <code>/k8s/deployment.yml</code> , <code>/k8s/service.yml</code>	Container build/run evidence, running pods/service, rolling update, and rollback.
Step 4 — Monitoring (2.0)	<code>/backend/src/api/main.py</code> , <code>/monitoring/prometheus.yml</code> , <code>/monitoring/docker-compose.yml</code>	Prometheus target UP, visible application metrics, and Grafana dashboard.
Step 5 — Report & Slides (1.5)	<code>/QuantShield_DevOps_CA2.pptx</code> and report	4–5 presentation slides plus technical documentation.
Step 6 — Bonus	External challenge proof, if completed	Submission, leaderboard, or GitHub proof.

Recommended Evidence Naming

```
devops_evidence/
├── task1_github_actions/
├── task2_ansible/
├── task3_docker_k8s/
├── task4_monitoring/
└── task5_report/
```

- Capture the GitHub Actions success screen and keep the workflow file in the repository.
- Capture the Ansible execution result with ok/changed/failed counts and evidence of the resulting configuration.
- Capture Docker build/run, Kubernetes pods/service, rolling update status, rollout history, and rollback verification.
- Capture the Prometheus targets page, a representative metrics query, and the Grafana dashboard.
- Keep the final report and presentation alongside the implementation artifacts or clearly linked from the README.

9. Artifact Index

Artifact	Purpose
.github/workflows/ci-cd.yml	Defines the GitHub Actions CI/CD workflow.
ansible/playbook.yml	Declarative configuration of the QuantShield runtime environment.
ansible/inventory	Defines the Ansible target host(s).
Dockerfile	Build instructions for the QuantShield FastAPI container image.
k8s/deployment.yaml	Kubernetes Deployment, replicas, probes, and rollout behavior.
k8s/service.yaml	Stable network endpoint for the QuantShield backend.
backend/src/api/main.py	FastAPI entry point and application metrics instrumentation.
monitoring/prometheus.yml	Prometheus scrape configuration.
monitoring/docker-compose.yml	Local orchestration for the monitoring stack.
QuantShield_DevOps_CA2.pptx	4–5 slide presentation covering architecture, pipeline, challenges, and lessons.

Conclusion

The QuantShield DevOps implementation demonstrates how an existing AI-driven application can be prepared for repeatable delivery and controlled operation without rewriting its core analytical logic. GitHub Actions provides the automated validation path, Ansible captures environment state as code, Docker establishes a reproducible runtime, Kubernetes manages service availability and release transitions, and Prometheus/Grafana provide visibility into runtime behavior.

Taken together, these components form a coherent lifecycle rather than a collection of isolated tools: changes are validated, packaged, deployed, observed, and—when necessary—reverted. That lifecycle is the primary outcome of the DevOps exercise and provides a practical foundation for extending QuantShield toward more automated deployment and production operations.

End of Report